병원 및 환자의 상태를 고려한 구급차 경로 최적화

Ambulance Route Optimization Considering Hospital and Patient Conditions

본 연구는 응급 환자 이송 과정에서 발생하는 '응급실 뺑뺑이' 문제를 해결하기 위해 효율적인 구급차 경로 최적화 시스템을 개발하였다. 병상 수, 의료진 인원, 혼잡도 등의 병원 공공데이터를 기반으로 데이터셋을 구축하고, 환자 중증도 및 시간대별 이송 분포를 반영한 시뮬레이션을 수행하였다. CP-SAT Solver를 이용해 제약조건을 만족하면서 이송 시간을 최소화하는 알고리즘을 구현하였으며, 이를 통합 플랫폼 "HeartLink"로 실시간 동기화 및 시각화하였다. 본 연구는 응급의료체계의 효율성과 환자 생존율 향상에 기여할 수 있는 기술적 기반을 제시하였다.

[주제어: 응급 의료 서비스, 스케줄링 시스템, 환자 위험도 계산, 병원 매칭]

Ⅰ. 서론

구급차로 환자를 이송하는 과정에서는 다양한 문제들이 발생한다. 그중 대표적인 사례가 바로 '응급실 뺑뺑이' 현상이다. 이는 구급차가 환자를 병원에 이송하려 하지만, 해당 병원의 응급실이 이미 만원 상태이거나 수용 한계를 초과하여 환자를 받아주지 못하는 상황을 의미한다. 이 과정에서 소중한 시간이 낭비되고, 환자의 상태가 급격히 악화될 위험이 있으며, 최악의 경우 환자가 이송 도중 사망할 가능성도 존재한다. [1][2]

현재까지도 이러한 피해는 반복적으로 발생하고 있으며, 이는 응급 의료 서비스의 효율성을 현저히 감소시키는 심각한 문제이다. 특히 현행 응급 의료 시스템은 경험과 직관에 의존하는 경우가 많아, 정량적인 데이터 분석에 기반한 의사결정이 부족하다.

이에 본 연구에서는 병원의 혼잡도와 환자의 상태를 정량적으로 분석하여, 환자를 가장 적합한 병원으로 신속히 이송할 수 있는 프로그램을 개발하고자 한다. 기존 시스템의 한계는 모든 병원 상황과 환자 특성을 동시에 고려하지 못한다는 점이다. 따라서 본 연구의 목적은 다음과 같다.

1. 통합 시스템을 구축하여 모든 환자를 최적의 병원으로 이송할 수 있도록 한다.
2. 응급 의료 체계 전반의 효율성을 향상시킨다.
3. 응급 상황에서 대응 시간을 단축시켜 응급 환자의 생존 가능성을 높인다.

Ⅱ. 이론적 배경

1. 기존 의료 시스템

1.1 응급환자 병원 이송 기준

1) 환자의 상태(중증도):

- 중증응급환자 -> 권역응급의료센터, 전문센터로 직접 이송을 고려
- 경증 환자 -> 가까운 지역응급의료기관, 응급실이 있는 종합병원 등으로 이송

1) https://news.kbs.co.kr/news/pc/view/view.do?ncd=8340709&ref=A
2) https://www.joongang.co.kr/article/25321432

2) 시간요소: 골든타임을 고려하여 특정 치료가 필요한 경우 가장 빨리 치료 가능한 병원을 선정
ex) 심근경색 -> 90분 이내 PCI(심장혈관 시술) 가능한 병원으로 이송
3) 병원의 가용성: 응급실 포화 상태 여부, 관련 전문 의료진 대기 여부
4) 거리와 접근성: 환자가 발생한 위치에서 이송 가능 시간이 가장 짧은 병원 우선, 도로 상황, 교통 체증에 따라 헬기 이송 여부 고려
5) 전문센터 지정 여부: 보건복지부 지정 병원체계에 따라 분류: 권역응급의료센터, 지역응급의료센터, 지역응급의료기관 등
6) 119 상황실/응급의료정보센터 연계: 119 구급대는 환자 상태를 보고하고, 응급의료정보센터에서 실시간으로 병원 가용 여부 확인 후 가장 적합한 병원 지정

1.2 기존 종합 의료 시스템
1) 종합상황판: 지역이나 병원을 검색하면 중환자실과 응급실 여부를 확인하고 어떤 환자가 있는지를 보여주는 시스템, 또한 중증응급질환 중 어떤 중증을 치료할 수 있는지를 알려준다.
2) KTAS(한국형 응급환자 분류도구): 응급환자의 증상에 따라 총 5개의 단계로 나누고 진료우선순위를 정하는 기준이다.

	요약 설명
KTAS1	즉각 처치 필요, 생명/사지 위협
KTAS2	빠른 치료 필요, 잠재적 위협 존재
KTAS3	생태 악화 가능성, 주의 관찰 필요
KTAS4	1~2시간 내 처치 가능, 경미한 증상
KTAS5	응급 아님, 민성/악화 가능성 작음

3) 국가응급의료자료:
① 의료기관 자원정보: 실시간 병상, 중증응급환자 수용가능현황(위의 종합상황판 이용)
② 국가응급 진료정보망: 표준등록체계, 심층등록체계, 외상등록체계(자료 이용신청 가능)
③ 응급의료 자원정보: 구급차 등록현황, AED 설치 현황
④ 출동 및 처치 기록지: 119 구급이송, 민간 구급이송, 응급전용헬기 구급이송
⑤ 응급의료 기관평가 정보

2. 스케줄링 알고리즘

2.1. FIFO(First Come First Served, First In First Out)
선진입선처리 알고리즘은 먼저 요청된 작업이 먼저 처리되는 방식으로 처리중인 요청이 있다면 해당 요청이 끝날 때까지 나머지 요청은 대기하는 방식으로 동작한다. 이 방식은 중간에 긴 시간을 요구하는 요청이 존재한다면 나머지 요청의 대기시간이 늘어난다는 문제점이 존재한다.

2.2. SJF(최소 작업 우선 스케줄링, Shortest Job First)
최소 작업 우선 스케줄링 알고리즘은 요청 중 가장 짧은 시간을 요구하는 것을 우선적으로 처리하는 방식이다. 이때 SJF는 선점형과 비선점형으로 나뉘게 된다. 이 방식은 대기시간은 짧지만, 긴 시간을 요구하는 작업이 짧은 작업에 지속적으로 밀릴 수 있다는 문제점이 존재한다.

2.3. 우선순위 스케줄링
우선순위 스케줄링은 각 요청에 대해 우선순위를 계산하여 우선순위가 높은 요청을 우선적으로 처리하는 방식으로 동작한다. 우선순위가 낮은 요청이 높은 요청에 의해 지속적으로 실행이 되지 않는 현상을 막기 위해 시간에 따라 점진적으로 해당 요청의 우선순위를 증가시키는 aging 기법

을 활용하기도 한다. 이 방식은 우선순위가 높은 요청이 지속적으로 들어온다면 우선순위가 낮은 요청은 처리가 이루어지지 않을 수 있다는 문제점이 존재한다.

2.4. 라운드 로빈 스케줄링(Round-Robin)

라운드 로빈 스케줄링은 요청에 대해 단위 시간인 할당량을 정의하여 그 시간만큼 자원을 할당하는 방식으로 해당 시간 안에 처리가 완료되지 않는다면 다음 요청이 자원을 할당받게 된다. 이 방식은 Context switching이 자주 발생하여 비용이 증가할 수 있고, 본 연구에서는 이송을 멈추었다가 다른 환자의 이송을 진행할 수 없는, Context switching을 진행할 수 없는 상황이기에 적용하기 어렵다는 문제점이 존재한다.

기존의 스케줄링 알고리즘들은 응급 이송 상황에서 병원 자원과 환자 상태를 동시에 고려하기 어렵다는 한계가 있었다. 이에 본 연구에서는 이러한 한계를 보완하기 위해, 이송 병원 자원 가용성과 환자 중증도 등을 통합적으로 반영하는 독자적인 우선순위 스케줄링 기법을 개발하여 적용하였다.

Ⅲ. 연구 방법 및 절차

1. 이송 병원 우선순위 스케줄링 알고리즘 개발

1.1. 데이터셋 구축

1) 병원 목록 수집

본 연구에서는 다음의 **응급 의료체계 관련 공공데이터를 활용하여 병원 정보를 구축하였다.** [3]비상진료 병의원 데이터를 통해 병원 목록을 추출하여 응급실을 보유한 병원의 목록을 축출하였고, [4]공공데이터 포털로부터 병원 기본 정보 및 진료 과목, 병상 수 등 세부 데이터을 수집하여 병원 목록과 연계하여 각 병원의 속성을 보완하였다. 이를 통해 이송 병원 우선순위 스케줄링 알고리즘 개발에 필요한 통합 병원 데이터셋을 구축하였다.

	골든 타임	중증도	질환 유형	진단 시간	발생 요일	환자 위치	병원	병원 위치
변수명	G_i	S_i	P_i	D_i	t_i	d_i	L_i	H_i

총 병상수	의료진 수	가용 병상	가용 의료진	응급실 병상	응급 병상	결정 변수
B_h^{total}	M_h^{tatal}	B_h^{avail}	M_h^{avail}	W_h	E_h	$x_{i,h}$

2) 테스트용 응급 이송 예시 데이터 생성

시스템을 테스트하기 위한 응급 이송 예시 데이터를 생성하였다. 실제 의료 현장과 비슷한 환경을 재현하기 위해 실제 구급차 응급 이송 통계 데이터와 인구 밀도 데이터를 확인하였다. 환자가 발생한 위치를 생성하기 위해 인구 밀도와 평균적인 응급 이송 건수가 비례하는 관계를 가진다는 점을 이용해 인구 밀도 맵을 바탕으로 가중치가 반영된 랜덤한 위치를 생성하였다. 또한 2025년도 119구급서비스 통계연보에 제시된 시간대별 응급 이송인원 분포를 적용하여 실제 환자들의 응급 신고 분포를 재현하였다.

3) https://www.e-gen.or.kr/egen/holiday_medical.do?emergencyViewYn=Y
4) https://www.data.go.kr/data/15000563/openapi.do

1.2. 목적 함수 정의

목적 함수에서 고려하는 변수는 다음과 같다.

G_i : i 번째 환자의 골든타임 (분)
S_i : i 번째 환자의 중증도 (1 ~ 5)
P_i : i 번째 환자의 질환 유형
D_i : i 번째 환자의 진단/트리아지 시간 (분)
t_i : i 번째 환자의 발생 시각
d_i : i 번째 환자 발생 요일
L_i : i 번째 환자의 위치 (위도, 경도)
H_h : h 번째 병원
Loc_h : h 병원의 위치 (위도, 경도)
B_h^{total} : h 병원의 총 병상 수
M_h^{total} : h 병원의 총 의료진 수
B_h^{avail} : h 병원의 가용 병상 수 (실시간)
M_h^{avail} : h 병원의 가용 의료진 수 (실시간)
W_h : h 병원의 응급실 병상 수
E_h : h 병원의 응급실 병상 수
$x_{i,h} \in \{0,1\}$: i 번째 환자의 h 병원 배정 여부

총 15개의 변수를 고려하였고, 이를 바탕으로 각 환자에 대한 배정 정보인 $x_{i,k}$를 계산하도록 하였다.

$$f_1(R_i,h) = S_i \begin{cases} 1 & (R_{i(h)} < 0) \\ 0 & (R_{i(h)} \geq 0) \end{cases}$$

$$f_2(R_i,h) = \begin{cases} S_i|R_i(h)| & (R_i(h) < 0) \\ S_i R_i(h) \times 0.5 & (0 \leq R_i(h) < 0.2G_i) \end{cases}$$

$$Match_i(h) = 1 - C_h$$

$$N_h = \sum_{i=1}^{n} x_{i,h}, \quad \overline{N} = \frac{n}{m}$$

$$f_3 = \frac{1}{m} \sum_{h=1}^{m} (N_h - \overline{N})^2$$

변수들을 바탕으로 목적함수 f_1, f_2, f_3를 정의하였다. 또한 각 변수들이 만족해야 하는 제약 조건도 아래와 같이 설정하였다.

$$\sum_{i=1}^{n} x_{i,h} \leq B_h^{avail}, \forall h \in H \qquad \sum s_{h=1}^{m} x_{i,h} = 1, \forall i \in I \qquad x_{i,h} \in \{0,1\}, \forall i \in I, \forall h \in H$$

또한 병원의 혼잡도를 계산하여 대기 시간을 예측하기 위해 환자의 중증도와 시간대 등의 값에 가중치를 곱한 형태로 예측 모델을 정의하였다. 이 과정에서 중증도와 시간, 요일은 실수 형태의 값으로 보정하여 사용하였다.

$$C_h = \alpha \frac{B_h^{total} - B_h^{avail}}{B_h^{total}} + \beta \frac{M_h^{total} - M_h^{avail}}{M_h^{total}} + \gamma \min(1.0, \frac{W_h}{10}), 0 \leq C_h \leq 1 \quad \text{(혼잡도)}$$

$$T_i(h) = KakaoMapAPI(L_i, Loc_h, t_i)$$

$$W_i(h) = f_W(C_h, B_h^{avail}, M_h^{avail}, S_i, t_i, d_i) W_{\cap}(h) = \sum_{j=0}^{J} \theta_j \psi_j(C_h, B_h^{avail}, M_h^{avail})$$

$$\psi_0 = 1, \psi_1 = C_h, \psi_2 = C_h^2 \psi_3 = \frac{1}{B_h^{avail}}, \psi_4 = \frac{1}{M_h^{avail}}, \psi_5 = \frac{C_h}{B_h^{avail}}, \psi_6 = \frac{C_h}{M_h^{avail}}$$

$$m_{severity}(S_i) = \begin{cases} 0.5 & (S_i = 5) \\ 0.7 & (S_i = 4) \\ 1.0 & (S_i = 3) \\ 1.3 & (S_i = 2) \\ 1.5 & (S_i = 1) \end{cases} m_{time}(t_i) = \begin{cases} 1.5 & t_i \in [22,6) \\ 1.2 & t_i \in [12,14) \\ 1.0 & t_i \notin (22,6] \text{ and } t_i \notin (12,14] \end{cases}$$

$$m_{day}(d_i) = \begin{cases} 1.3 & d_i \in \{Sat, Sun\} \\ 1.0 & d_i \notin \{Sat, Sun\} \end{cases}$$

$$W_i(h) = W_{\text{base}}(\text{h}) \times \text{m}_{\text{severity}}(\text{S}_\text{i}) \times \text{m}_{\text{time}}(\text{t}_\text{i}) \times \text{m}_{\text{day}}(\text{d}_\text{i}) \quad \text{(병원 대기시간)}$$

$$A_i(h) = T_i(h) + W_i(h) + D_i \quad \text{(처치 전까지 소요되는 총 시간)}$$

$$R_i(h) = G_i - A_i(h) \quad \text{(이송 후 남는 골든타임)}$$

스케줄링 알고리즘을 통해 도출해낼 최적의 이송 병원 배치는 다음과 같이 정의된다.

1. 골든타임을 초과하는 환자의 인원수가 최소가 되어야 한다.
2. 각 환자가 병원에 도착하여 처치가 진행되는 시점에 남는 골든타임이 최대가 되어야 한다.
3. 각 환자가 구급차를 통해 이송되는 시간이 최소가 되어야 한다.
4. 병원의 부하가 분산되어야 한다.

이때 위 4가지 조건이 1번, 2번, 3번, 4번의 우선순위로 고려해야 한다. 따라서 알고리즘이 각 조건을 나타내는 함수 S1, S2, S3, S4를 순서대로 최적화할 수 있도록 lexmin을 사용하여 아래와 같이 하나의 최종 목적 함수 S를 정의하였다.

$$\min S_1 = \sum_{i=1}^{n}\sum_{h=1}^{m} f_1(R_i,h)x_{i,h} \quad \text{(골든타임 초과 최소화)}$$

$$\max S_2 = \sum_{i=1}^{n}\sum_{h=1}^{m} [R_i(h)Match_i(h)]x_{i,h} \quad \text{(여유시간} \times \text{매칭점수 최대화)}$$

$$\min S_3 = \sum_{i=1}^{n}\sum_{h=1}^{m} T_i(h)x_{i,h} \quad \text{(이동시간 최소화)}$$

$$\min S_4 = f_3 \quad \text{(병원간 부하 균형)}$$

$$S = \text{lexmin}(S_1, -S_2, S_3, S_4)$$

lexmin으로 S_1, S_2, S_3, S_4를 묶는 형태로 정의하였고, 이 중 S_2만 최대화해야 하므로, S_2에 -1을 곱하여 $-S_2$를 최소화하는 것으로 변환하였다. 이때 여유시간과 매칭점수를 분리하여 lexmin을 적용하지 않고, 곱의 합을 계산하여 하나의 함수로 lexmin에 적용한 이유는 여유시간과 매칭점수 모두 공통적으로 병원 도착 후의 시간과 관련이 있기 때문이다. 만약 이 두 변인 중 하나를 더 높은 우선순위에 배치할 경우 병원에 도착한 후 남는 골든타임은 길지만 병원이 과도하게 혼잡하여 결과적으로 처치가 제대로 이루어지지 않거나, 병원은 혼잡하지 않아 도착 시 바로 처치가 가능하지만 도착 후 남는 골든타임이 부족하여 처치를 위한 시간이 부족하는 등의 예외 케이스가 발생할 가능성이 있다. 따라서 이 두 변인을 하나의 값으로 혼합하기 위해 둘의 곱의 합의 형태로 정의하였다.

1.3. CP-SAT Solver를 이용한 스케줄링

최적의 이송 병원의 배치의 탐색은 앞에서 정의한 목적 함수 S를 최소화하는 최적화 문제로 생각할 수 있다. 이를 해결하기 위한 방법은 Backtracking, Forward Checking, Arc Consistency (AC-3), Min-Conflicts, Branch & Bound, CP-SAT, Integer Programming (ILP/LP) 등이 존재하는데, 의료 시스템에서는 정확성이 속도보다 더 중요하게 작용한다는 점을 고려하여 CP-SAT 방식을 선택하였다. 이에 따라 제약조건 내에서 목적함수 S를 최소화 할 수 있는 변수들의 값을 탐색하기 위해 CP-SAT Solver를 이용하여 병원 배치 탐색 시스템을 구현하였다.

```
# 각 환자는 정확히 하나의 병원에 배정
for i in range(n_patients):
    model.Add(sum(x[(i, h)] for h in range(n_hospitals)) == 1)

# 병상 용량
for h in range(n_hospitals):
    model.Add(sum(x[(i, h)] for i in range(n_patients)) <= hospitals[h].available_beds)
```

[Fig. 1] CP-SAT 제약조건 설정 소스코드

최적해를 계산하기 전에 위에서 정의한 변수들이 항상 만족해야 하는 제약조건을 설정하였다.

```python
@property
def congestion(self) -> float:
    weights = self._predictor.congestion_weights

    bed_usage = (self.total_beds - self.available_beds) / self.total_beds if self.total_beds > 0 else 0
    staff_usage = (self.total_staff - self.available_staff) / self.total_staff if self.total_staff > 0 else 0
    waiting_factor = min(1.0, self.waiting_patients / 10)

    congestion = (weights['bed'] * bed_usage +
                  weights['staff'] * staff_usage +
                  weights['waiting'] * waiting_factor)
```

[Fig. 2] 병원 혼잡도 계산 소스코드

각 병원의 정보를 담고 있는 class에 대해 혼잡도를 계산하도록 구현하였다. 학습된 예측 모델로부터 가중치 목록을 불러온 후, 이를 실시간 사용중인 병상수, 처치중인 의료진 수, 대기 환자 수에 곱하여 최종적인 혼잡도(congestion)을 계산하였다.

```python
def trigger_retraining(self):
    if len(self.wait_predictor.feedback_data) >= 20:
        print("\n" + "="*50)
        print("Sufficient Feedback Data Collected - Starting Retraining")
        print("="*50)
        self.wait_predictor.retrain(learning_rate=0.001)
        self.wait_predictor.save()
```

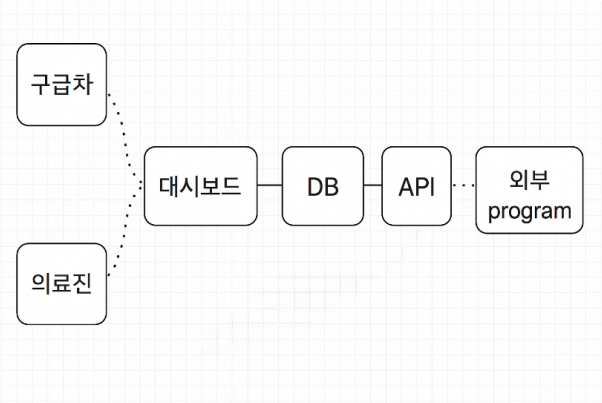
[Fig. 3] 병원 혼잡도 예측 모델 피드백 소스코드

또한 병원의 혼잡도를 기반으로 대기시간을 예측하는 모델이 응급 신고 경향성의 변화 변화에 대응하여 최적의 배치를 계산하기 위한 적절한 가중치를 유지할 수 있도록, 예측에 따라 생성된 환자들의 병원 배치의 결과를 피드백하는 시스템을 적용하였다. 이에 따라 주기적으로 모델 내부에서 경사하강법을 통해 0.001의 learning reate로 모델 가중치가 조정되고, 이것이 새로운 예측에 반영된다.

모델의 초기 가중치의 경우 생성한 테스트용 응급 이송 예시 데이터를 통해 가상의 응급 이송 상황을 시뮬레이션하여 모델이 학습을 통해 가중치를 설정할 수 있도록 하였다.

2. “HeartLink” 통합 시스템 개발

기존에 병원에서 활용되고 있는 시스템과 연동하여 활용할 수 있도록 이 시스템에 응급 환자를 등록하고, 등록된 환자나 환자들의 병원 배치 상태, 이송 현황 등을 확인할 수 있는 API를 개발하였다. 시스템 구조는 아래와 같이 설계하였다.



[Fig. 4] 시스템 구조 다이어그램

제작한 시스템을 실제로 활용할 수 있는 웹 클라이언트를 개발하였다. 클라이언트는 구급차에서 환자의 정보를 기록하고, 매칭된 병원을 확인할 수 있는 구급차용 클라이언트와, 구급차를 관리하고 환자의 실시간 이송, 처치 정보를 확인할 수 있는 병원용 클라이언트로 나뉜다.

2.1. 계정 인증 시스템 개발

병원 의료진과 구급차를 구분하고, 의료 데이터 접근 권한이 없는 외부자의 접근을 막기 위해 계정 인증 시스템을 개발하였다. 병원 의료진의 경우 이메일과 비밀번호를 통해 로그인이 가능하도록 하였고, 구급차의 경우 발급받은 구급차 ID를 통해 로그인이 가능하도록 하였다. 이를 위해 RDBMS 중 하나인 PostgreSQL을 이용하여 DB를 구축하였고, JWT 기반의 인증 시스템을 구현하였다. 이를 기반으로 인증 토큰이 없는 client의 의료 정보 페이지 접근을 차단하였다.

추가적으로 client 변조 등을 통한 우회 API 호출로 인한 데이터 유출, 수정, 소실을 방지하기 위해 서버로 들어온 SQL 실행문에 DB 접근 범위 제한을 두는 Policy를 적용하였다.

2.2. 구급차 대시보드 개발

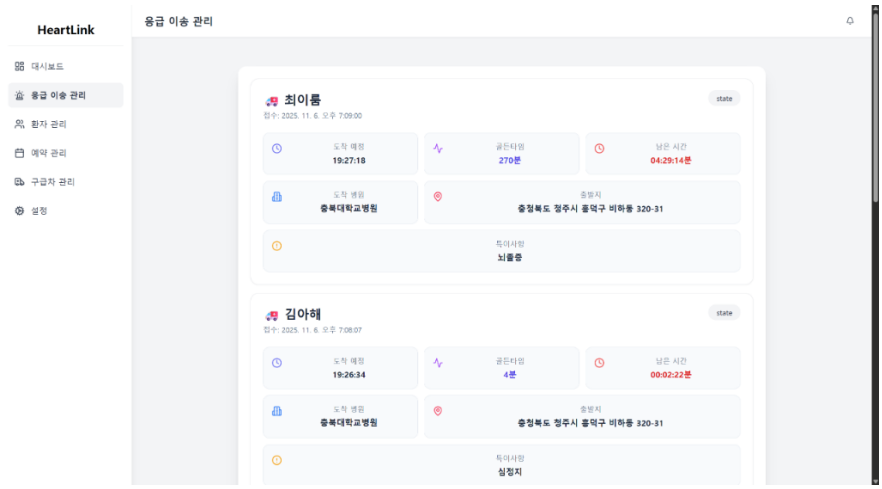
구급차에서 대시보드를 통해 현재 이송중인 환자에 관한 정보와 이송 병원 정보, 골든타임 정보 등을 확인할 수 있는 대시보드를 개발하였다. 구급차 대시보드에서 이송 정보 관련 데이터베이스의 update이 발생할 경우 화면을 자동으로 갱신하도록 하기 위해 데이터베이스의 이송 정보 table에 update를 모든 client에게 broadcast하는 channel을 생성한 후 구급차 대시보드 client에서 이를 subscribe하도록 하여 화면 자동 갱신을 구현하였다.



[Fig. 5] 구급차 대시보드 화면

2.3. 병원 의료진 대시보드 개발

병원의 의료진이 해당 병원으로 이송중인 환자들과 이송이 완료된 환자들의 정보를 확인할 수 있는 대시보드를 개발하였다. 응급 이송 관리 목록 메뉴를 통해 해당 병원으로 이송중이거나 이송이 완료된 환자들의 병원 도착 예정 시간과, 골든타임이 끝날 때까지 남은 시간, 응급 상황 발생 위치, 특이사항이 표시된다.



[Fig. 6] 병원 의료진 대시보드 화면

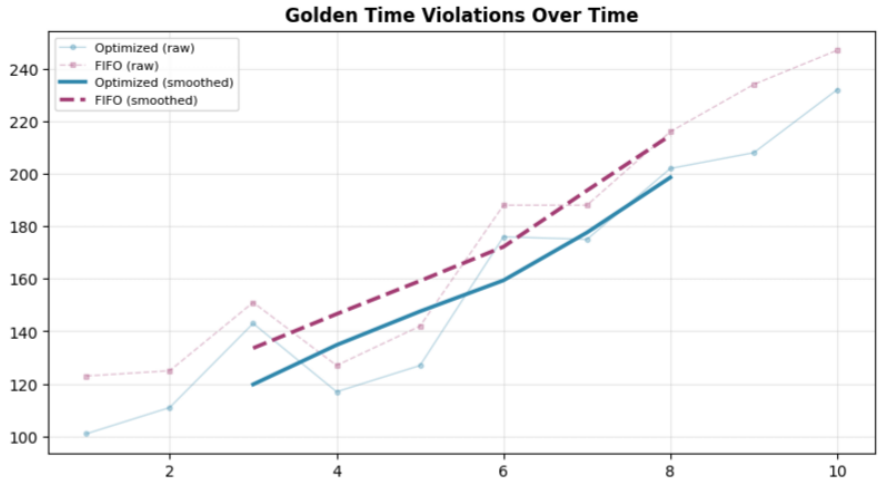
Ⅳ. 연구 결과

본 연구에서는 기존의 FIFO(First-In-First-Out) 방식 스케줄링 알고리즘과 새롭게 설계된 Optimized Scheduler 알고리즘을 동일한 시뮬레이션 환경에서 비교 분석하였다. 그 결과, 제안한 Optimized Scheduler는 기존 방식에 비해 전반적인 응급환자 이송 및 병원 배정 효율성을 유의미하게 향상시켰음을 확인하였다.

먼저, 골든타임 위반(중증 환자가 시간 내 처치를 받지 못한 경우)의 가중 합은 FIFO 방식에서 352건이었던 반면, Optimized Scheduler에서는 335건으로 약 4.8% 감소하였다. 이는 알고리즘이 단순한 선착순 처리 대신 환자의 중증도와 병원 수용 가능성을 함께 고려하여, 보다 시급한 환자가 우선적으로 배정되도록 조정했기 때문이다. 이러한 결과는 실제 응급의료 현장에서 생명 유지율을 향상시킬 가능성이 있음을 시사한다. 또한, 골든타임을 넘긴 환자 수 역시 127명에서 121명으로 감소하여, 환자 개개인의 치료 접근성이 개선되었음을 보여준다.

이송 효율 측면에서도 개선이 뚜렷하게 나타났다. 총 이송 시간(Transport Time)은 FIFO의 3813.0분 대비 Optimized Scheduler에서 3532.0분으로 약 8.0% 단축되었다. 이는 각 구급차의 이동 경로를 단순 직선 거리 기반이 아닌, 병원의 실시간 수용률과 환자 중증도를 함께 고려하여 최적화한 결과이다. 그에 따라 불필요한 이동이 줄어들고, 동일 시간 내 더 많은 환자를 이송할 수 있게 되었다.

한편, 병원 도착 이후의 총 대기 시간(Wait Time) 역시 11773.6분에서 11303.0분으로 약 4.0% 감소하였다. 이 수치는 환자들이 병원 내 진료를 대기하는 시간이 줄었음을 의미하며, 응급실 과밀화 문제를 완화시키는 효과를 간접적으로 보여준다. 특히, 이러한 대기 시간 감소는 단순히 병원 배정 속도의 문제가 아니라, 시스템 전반의 자원 분배 효율성 개선에서 비롯된 결과로 해석할 수 있다.



[Fig. 7] 시간에 따른 골든타임 위반 환자 수

또한, 다양한 환경에서의 대응 가능성을 확인하기 위해 응급 환자 수, 처치 종류 등을 무작위루 설정하여 반복을 시행한 결과에서도 제작한 알고리즘이 피드백 과정을 통한 가중치 재조정을 통해 비교 대상인 FIFO 방식보다 지속적으로 더 향상된 배치를 도출해낼 수 있음을 확인하였다.

Ⅴ. 고찰

본 연구에서는 병원과 환자의 상태를 함께 고려하여 구급차 이송 경로를 효율적으로 결정하는 방법을 탐구하였다. CP-SAT 기반 제약 조건 프로그래밍을 활용해 환자 중증도, 병상 수, 의료진 수 등의 다양한 요소를 반영한 우선순위 스케줄링 알고리즘을 구현하였으며, 그 결과 단순 거리 기준보다 병원 혼잡도를 줄이면서 환자 이송 시간을 단축할 수 있음을 확인하였다.

실제 병원의 실시간 데이터들로 여러 데이터셋을 만든 후 이를 기반해 렌덤으로 각 개인마다의 데이터를 만들고 연구에서 만든 통합 시스템에 사용되었다. 이 시스템은 의학계열 또는 환자만 접근 가능하기에 계정 인증 시스템, 구급차로 이송중인 환자 정보, 골든 타임과 이송해야하는 병원을 알려주는 대시보드, 의료진이 환자들의 정보를 확인할 수 있는 대시보드를 개발하였다.

Ⅵ. 결론

1. 연구 요약

본 연구는 응급 환자 이송 과정에서 반복적으로 발생하는 '응급실 뺑뺑이' 문제를 해결하기 위한 효율적인 구급차 경로 최적화 시스템을 개발하는 것을 목표로 하였다. 기존의 응급의료체계는 경험적 판단에 의존하여 병원 수용 한계와 환자 중증도, 실시간 혼잡도를 충분히 반영하지 못하는 구조적 한계를 지니고 있었다. 이에 병원 혼잡도와 환자의 상태를 통합적으로 분석하여 최적의 병원을 자동으로 추천하는 시스템을 고안하였다.

연구 과정에서는 먼저 병원 및 환자 관련 공공데이터를 활용해 병원 목록, 병상 수, 의료진 인원 등의 속성을 포함한 데이터셋을 구축하였다. 이후 환자의 중증도와 시간대에 따른 응급 이송 분포를 반영하여 테스트 데이터를 생성하였다. 이 데이터를 기반으로 CP-SAT Solver를 이용해 제약 조건 하에서 목적함수를 최소화하는 최적화 알고리즘을 구현하였으며, 이를 통해 병원 혼잡도를 줄이면서 환자의 골든타임 내 이송률을 향상시키는 결과를 얻었다.

또한, 개발된 시스템은 "HeartLink"라는 이름의 통합 플랫폼으로 구현되었다. 이 플랫폼은 구급차와 병원 간 실시간 데이터 동기화를 지원하며, 구급차 대시보드에서는 환자의 이송 현황과 골든타임 정보를, 병원 대시보드에서는 도착 예정 환자 정보를 한눈에 확인할 수 있다. 계정 인증 시스템을 통해 데이터 보안을 강화하였으며, PostgreSQL 기반의 인증 구조와 JWT 토큰을 활용하여

접근 권한을 관리하였다.

2. 기대 효과

본 연구를 통해 개발할 시스템은 환자와 병원의 상태를 실시간으로 분석하여, 환자를 가장 적합한 병원으로 신속하게 이송함으로써 응급환자의 생존률을 높이고 전반적인 응급 처리 성공률을 향상시킬 것이다. 또한 병원의 혼잡도와 의료 자원의 가용성을 함께 고려하는 스케줄링 기반 최적화 시스템을 통해, 의료기관 간의 환자 분산 효과를 유도하고 불필요한 대기 및 중복 이송을 줄일 수 있을 것이다.

이를 통해 의료 자원의 활용 효율성이 확대되고, 각 병원 응급실의 과밀 현상 완화 및 대기 시간 감소가 기대된다. 더 나아가 의료진은 보다 합리적인 근무 배분이 가능해져 업무 피로도와 과중 부담이 경감될 것으로 예상된다. 또한 실시간 데이터 동기화를 기반으로 한 이송 체계는 향후 응급의료정보망과 연계되어 지역 단위의 응급 대응 시스템 개선으로 확장될 가능성이 있다. 결과적으로 본 연구는 응급의료체계 전반의 효율성과 안정성을 높이는 기술적 기반을 마련한다는 점에서 의의를 가진다.

■ 참고문헌

[1] Olivos C., Caceres H.,. (2022).「Multi-objective optimization of ambulance location in Antofagasta, Chile」
https://journals.vilniustech.lt/index.php/Transport/article/view/17073

[2] B. Andersson, S. Baruah, J. Jonsson. (2001).「Static-priority scheduling on multiprocessors」
https://ieeexplore.ieee.org/abstract/document/990610

[3] 이정민, 조성환, 김찬용, 김민정, 김지훈, 하성민, 장혁재. “응급환자 처치 및 이송을 위한 지능형 구급시스템 설계 및 고찰.”《대한전자공학회 학술대회 논문집》, 2024년 6월, pp. 277-279.
https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11890406

[4] 노혜지. (2024). 한국형 응급환자 분류도구와 응급의료관리료 제도의 응급판정기준 분석 [석사학위논문, 을지대학교 보건복지대학원]. 을지대학교 학술정보원.
https://www.dbpia.co.kr/journal/detail?nodeId=T16965103

[5] 박채경, 김민주, 전지호, 유승헌. (2023).「응급환자 긴급구조 효율 향상을 위한 구급 정보 통합 서비스 설계」. 한국디자인학회 학술발표대회 논문집
https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11566217

[6] 소방청. (2025). 2025년도 119구급서비스 통계연보(최종). 대전: 소방청.(발간등록번호 11-1661000-000014-10, 작성기준일 2024.12.31.)